Computer Vision Based Tree Modeling

Wyatt Barber
EGR 600 Semester Project

Problem Background....3
Image Processing Pipeline....4
Image Segmentation Model....4
Feature Identification....9
Feature Orientation....10
Analyzing Connectivity....11
Confidence Metric....13
Results....13
Reconstruction Accuracy....13
Confidence Metric....16
Appendix: Relevant CNN Definitions....18
References....20

Problem Background

Agriculture consists of numerous time-consuming tasks, requiring labor that is becoming increasingly scarce and expensive [1]. In orchards, automated technology has been developed for many tasks, such as harvesting and spraying, but others remain difficult to automate. Pruning is one of these. Due to the complicated and unpredictable structure of trees, as well as the complexity of pruning decisions, no automated pruning technology has been widely adopted [2]. This is an area where automation could bring significant benefits to orchard operations, as pruning is the second most time consuming task in orchard operations [1].

The complexity of automated pruning is not a problem limited to orchard management. It is an important task in power line maintenance as well; regular pruning of trees near power lines reduces the number of outages caused by falling or damaged branches. However pruning these trees poses hazards to workers from fall, injury by the tools they use, or electrical shock [3]. Automating this task reduces this risk, but requires a robot that can perform the task with awareness of a tree's structure and what the consequences of cutting a limb may be.

The goal of this project is to create an image processing pipeline that creates a graph-like model of a tree’s branch structure. This model could then be used to inform a robot's decisions such as how to move an end effector through a tree canopy, or where to prune. The model will be task-agnostic; the pipeline that creates it and the data the model contains will not be optimized or constrained for a specific task, tree type, or environment.

Image Processing Pipeline

Image Segmentation Model

The first stage of the processing is determining which pixels in the input image represent tree branches and which do not. To do this, a convolutional neural network (CNN) based on a pytorch tutorial [4] [5] is used. The model uses the SegNet architecture [6], described in figure 1. CNNs will not be described in detail in this paper, but some definitions of relevant terms can be found in the appendix.

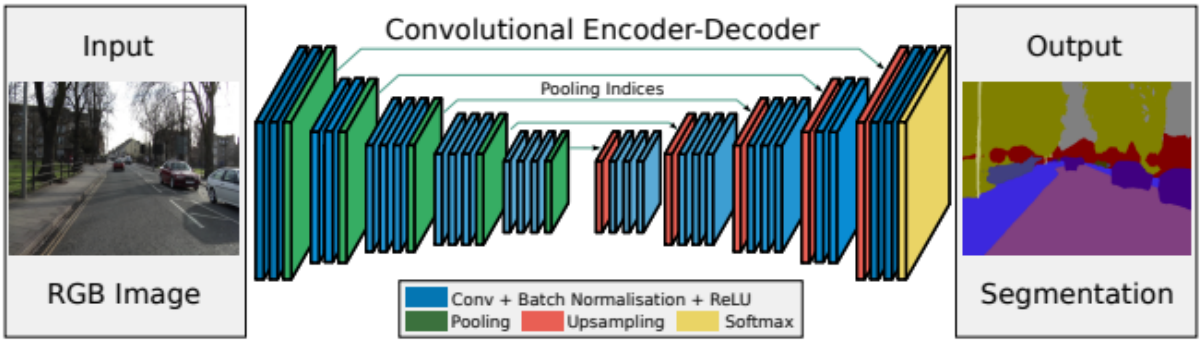


Figure 1: The SegNet image segmentation architecture
Source: [6]

To train a model for this task, a dataset containing 1,485 RGB images [7] of trees in urban environments paired with binary masks indicating what pixels are part of the tree's structure was used. Figure 2 shows example images and masks from this dataset.



Figure 2: Example 128x128 images from the urban street tree dataset

The exact specifications of the model used in this project can be seen in table 1. One notable difference from the SegNet architecture shown is the use of a sigmoid output layer instead of softmax. Softmax is used to produce K output channels classifying each pixel as most likely belonging to one of K classes. Since this case only has two output classes, tree or not tree, softmax is not necessary and the single channel sigmoid function is simpler and faster to use.

Table 1: Tree Segmentation CNN Architecture. All sizes given in pixels.

Layer Type	**Input Channels**	**Output Channels**	**Input Size**	**Output Size**
Batch Norm	3	3	128x128	128x128
Convolution	3	64	128x128	128x128
Batch Norm	64	64	128x128	128x128
ReLU	64	64	128x128	128x128
Convolution	64	64	128x128	128x128
Batch Norm	64	64	128x128	128x128
ReLU	64	64	128x128	128x128
Max Pool	64	64	128x128	64x64
Convolution	64	128	64x64	64x64

Batch Norm	128	128	64x64	64x64
ReLU	128	128	64x64	64x64
Convolution	128	128	64x64	64x64
Batch Norm	128	128	64x64	64x64
ReLU	128	128	64x64	64x64
Max Pool	128	128	64x64	32x32
Convolution	128	256	32x32	32x32
Batch Norm	256	256	32x32	32x32
ReLU	256	256	32x32	32x32
Convolution	256	256	32x32	32x32
Batch Norm	256	256	32x32	32x32
ReLU	256	256	32x32	32x32
Convolution	256	256	32x32	32x32
Batch Norm	256	256	32x32	32x32
ReLU	256	256	32x32	32x32
Max Pool	256	256	32x32	16x16
Convolution	256	128	16x16	16x16
Batch Norm	128	128	16x16	16x16
ReLU	128	128	16x16	16x16
Convolution	128	128	16x16	16x16
Batch Norm	128	128	16x16	16x16
ReLU	128	128	16x16	16x16
Convolution	128	128	16x16	16x16
Batch Norm	128	128	16x16	16x16
ReLU	128	128	16x16	16x16
Max Unpool	128	128	16x16	32x32
Convolution	128	64	32x32	32x32

Batch Norm	64	64	32x32	32x32
ReLU	64	64	32x32	32x32
Convolution	64	64	32x32	32x32
Batch Norm	64	64	32x32	32x32
ReLU	64	64	32x32	32x32
Max Unpool	64	64	32x32	64x64
Convolution	64	64	64x64	64x64
Batch Norm	64	64	64x64	64x64
ReLU	64	64	64x64	64x64
Convolution	64	1	64x64	64x64
Batch Norm	1	1	64x64	64x64
Sigmoid	1	1	64x64	64x64
Max Unpool	1	1	64x64	128x128

The model was trained for 21 epochs, with an initial learning rate of 0.001 and an Adam optimizer with decay rates $\beta_1 = 0.9, \beta_2 = 0.999$. Binary cross-entropy was used as the loss function. Roughly 10% of the dataset was withheld from training to be used for cross-validation. Figure 2 shows plots of the models performance during training.

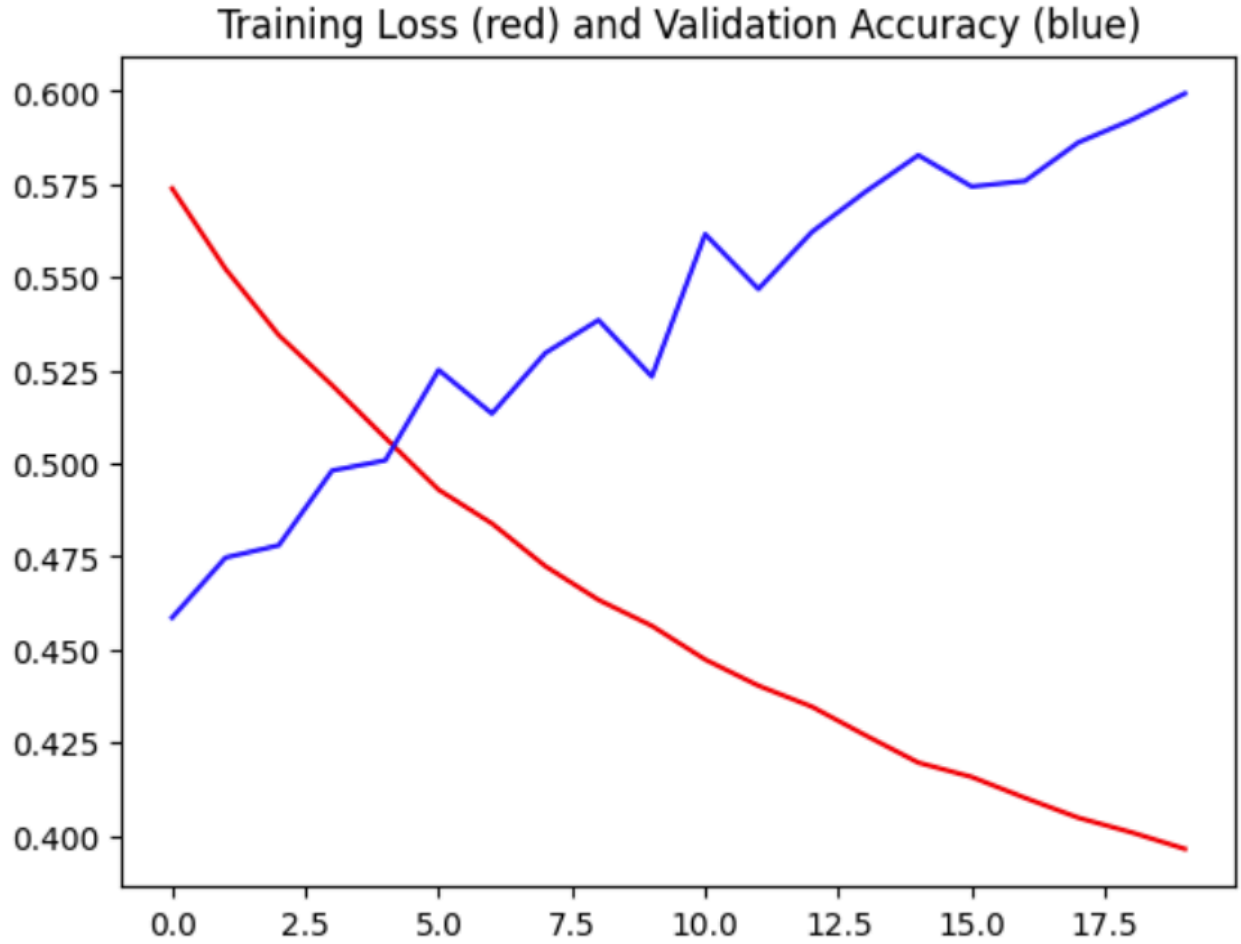


Figure 3: Model training loss and validation accuracy

These plots show that the model is learning, and as the blue line (validation accuracy) is still trending up the model is not overfitting. That both lines have not leveled out suggests that the model would benefit from more training, but tuning training time and model hyper parameters is outside the scope of this project.

Visual inspection of some of the validation images shows that the model is correctly generating masks that identify what parts of an image are tree.

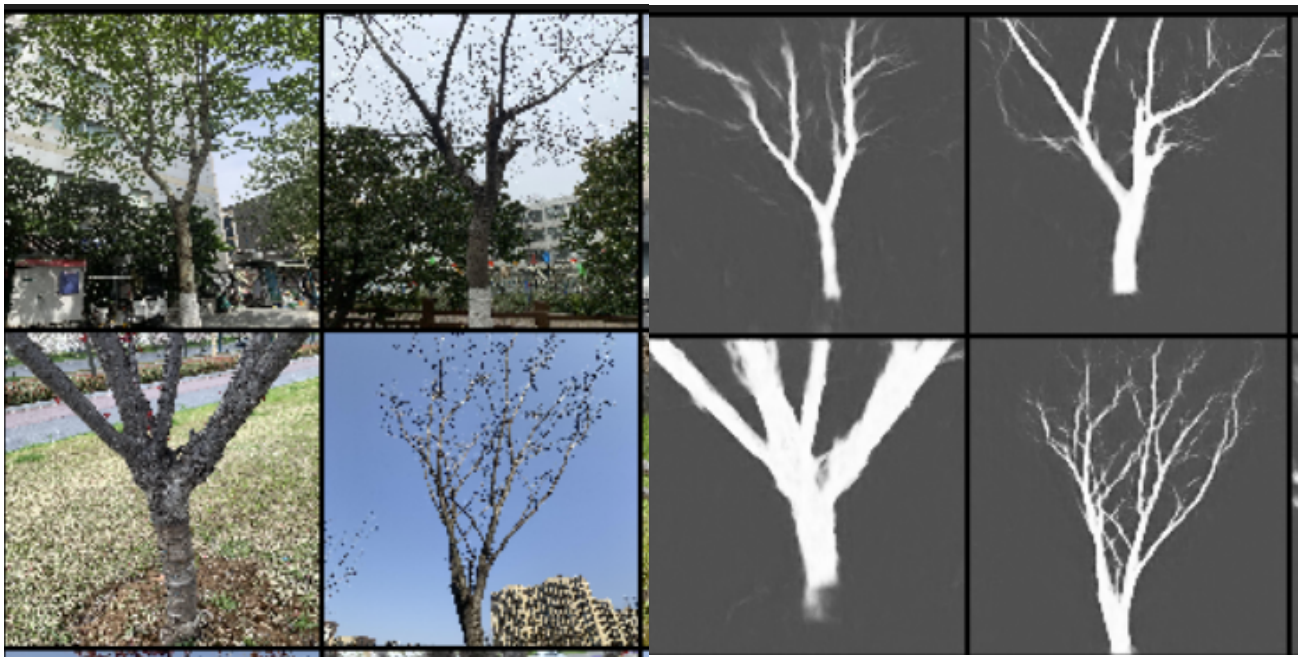


Figure 4: Four images from the dataset and the masks generated by this model after 21 epochs of training.

Feature Identification

This stage identifies points where the tree branches split or end. These points serve as the identifying features of the tree in this model, and make up the nodes of the graph modeling a tree.

To do this, first a binary mask is generated by thresholding the output of the CNN at 0.5. Pixels with an activation of greater than 0.5 are given a value of 1, and all others are given a value of 0.

Next this mask is skeletonized using scikit-image's [8] "skeletonize" function. Skeletonization takes a binary mask and converts all regions of "1" values to lines of 1 pixel width. This produces a mask that preserves just the basic structure of the input.

The skeleton is then used to identify nodes in the tree. A node is any pixel that meets both of the following criteria:

1. The pixel has a value of 1
2. The pixel has n neighbors with a value of 1, where $n \neq 2$

Pixels that meet this represent the center of a node: a point where a branch either splits ($n > 2$), or ends ($n = 1$). A single point ($n = 0$) also counts as a node, but ideally should not appear in the input.

Finally, the size of each node is estimated. From the center of the node, the distance in each of the directions up, down, left, and right is calculated to the first 0 valued pixel in the skeletonized mask. The median of these 4 values is set as the radius of the node. Figure 4 illustrates this process for a mask image taken from the training dataset.

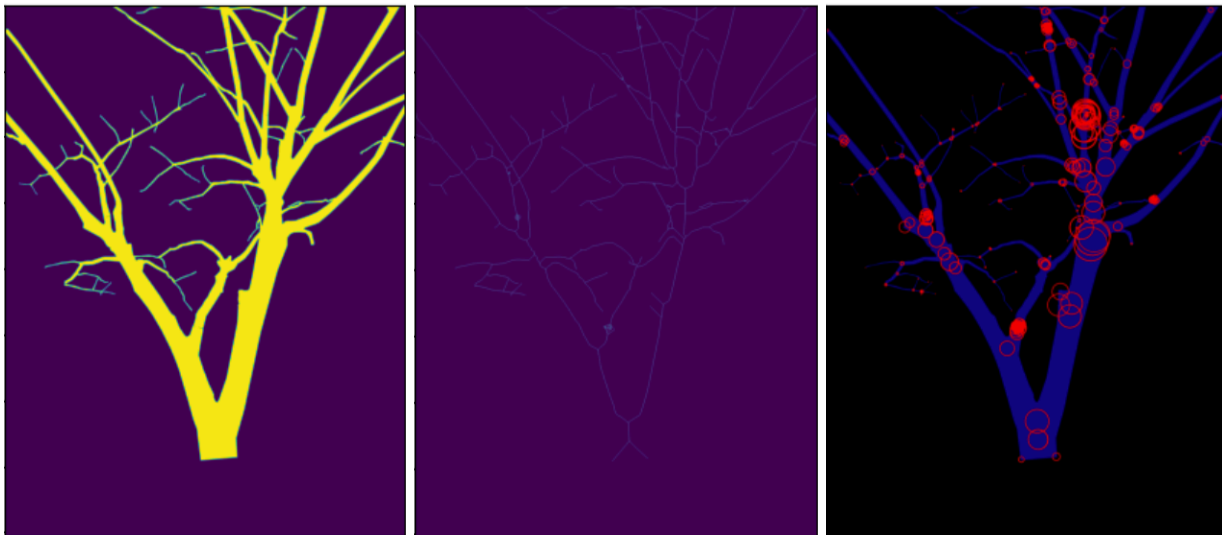


Figure 5: Input mask (left), skeleton (center), and mask with nodes (right)

Feature Orientation

At this stage, all the nodes identified previously are analyzed to determine the angle at which they are connected to their adjacent branches. To do this, a window of 9x9 pixels is isolated around each node. Then for each 1 valued pixel in the window an angle and distance are calculated as in equations 1 and 2, where:

- j_n is the width coordinate of pixel n
- i_n is the height coordinate of pixel n
- c_1 is the width coordinate of the nodes center
- c_0 is the height coordinate of the nodes center

$$\alpha_n = arctan2(j_n - c_1, c_0 - i_n)$$ Equation 1: Angle of a pixel from node center

$$r_n = \sqrt{(i - c_0)^2 + (j - c_1)^2}$$ Equation 2: Distance of a pixel from node center

Each pixel is assumed to approximate the true orientation of the branch, with some added error. To get all angles, each pixel is represented by a normal distribution with mean α and standard deviation r. Adding all these distributions together as in equation 3 produces a smooth density

function that allows us to identify the K most likely branch angles as the K highest peaks of the function, where K is the number of adjacent 1 valued pixels to the node center.

$$P(\theta) = \sum_{n}^{N} \frac{1}{2\pi r_n^2} e^{\frac{-(\theta - \alpha_n)^2}{2r_n^2}}$$ Equation 3: Branch angle density function

Figure 5 shows an example node and its density function. Angles are measured from 0° pointing straight up, to 360° rotating clockwise. The plot shows 3 peaks at roughly 100°, 230°, and 320°, which appears to match the orientation of lines in the image.

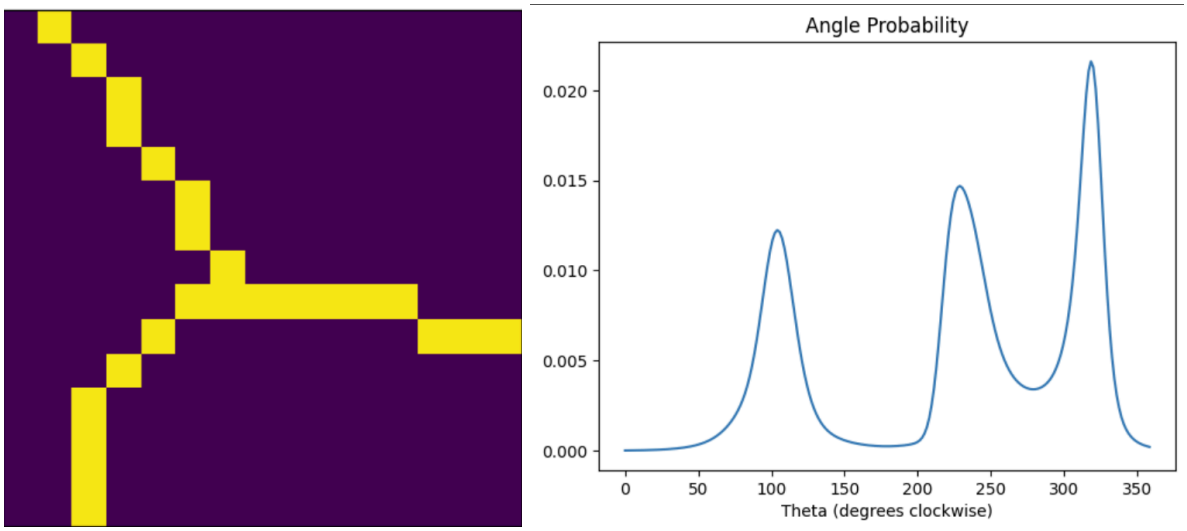


Figure 6: Node with three branches (left) and its probability density function showing 3 peaks (right). The plot is normalized to sum to 1, for readability.

The angles calculated in thes step ended up not being useful for the next stage as planned, but they were left in as they could still be useful were this method to be applied to a real problem.

Analyzing Connectivity

This is the final stage of assembling the model. Here each node is connected to its appropriate children. The method used here is rather naive, but simple to implement. For each node, the adjacent 1 valued pixels are identified. Then from that pixel, the algorithm walks down the line of connected pixels in the skeleton until another node location is found. That node is then registered as a child of the starting node. Figure 6 shows the reconstructed result of this process. The reconstruction is generated by drawing a line for each branch in the model between the two endpoint nodes. The line's thickness is the average of the two node's thickness.

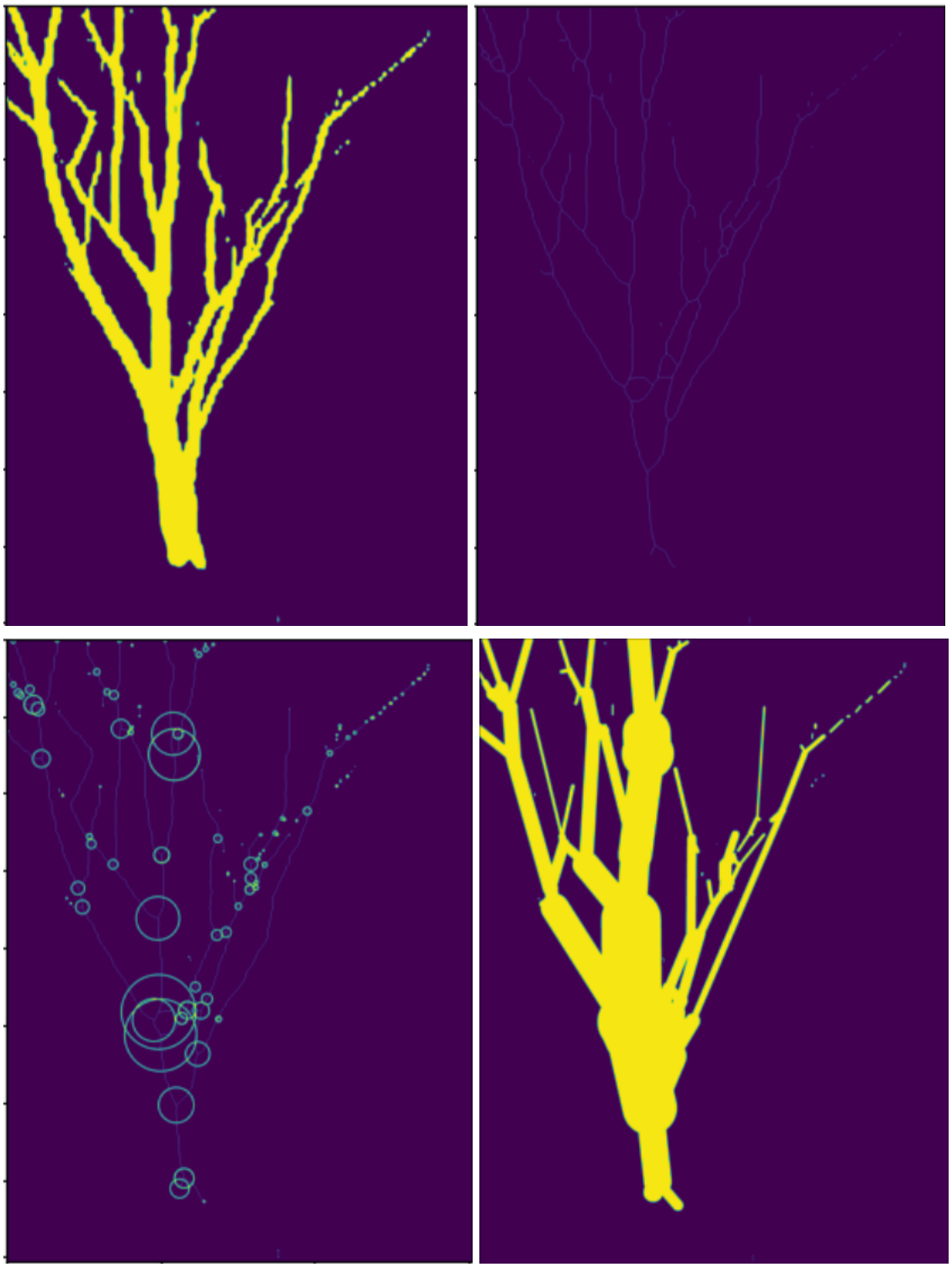


Figure 7: Complete modeling process, from CNN output (upper left), skeleton (upper right), node placement (lower left), to connection and reconstruction (lower right)

Confidence Metric

The models generated by this process should be able to score their own confidence as an indicator of the model's likelihood of being correct. In a real world application this would allow for automating decisions such as whether more images must be collected to create a better model, or whether a robot should proceed with a task or wait for human intervention.

Since the model does not have the true mask to compare its reconstruction against, it has to gauge its confidence against assumptions about the structure of a tree. The assumption used in this project is that all branches in a tree are smaller than the branch they grew from. This means that all nodes should have a greater width than their children.

To implement this, each node scores its confidence using equation 4, where:

- w is the width of the current node
- w_n is the width of the nth child node
- N is the number of child nodes connected to the current node

$$C = 1 - \frac{1}{N}\sum_{n}^{N}\frac{\sqrt{w^2 - w_n^2}}{w}$$ Equation 4: Confidence score for an individual node

The average confidence over all nodes is taken to get the complete model confidence. Nodes that have no children return a confidence of 1.

This confidence metric does assume that a perfect score would be a node where all its children are of width equal to it. This of course is not correct, there is some expected difference between connected nodes. But finding that difference is difficult, and this metric still gives a score that decreases as the difference in width between connected nodes increases.

Results

Reconstruction Accuracy

Analysis of model accuracy is done by reconstructing a binary mask for each model, as described previously. Then the accuracy compared to the training mask will be calculated with equation 5, where $reconstruction_n[i,j]$ and $test_n[i,j]$ are the mask values (0 or 1) at coordinates i,j in the nth image in the dataset.

$$A_n = 1 - (\sum_{i,j}|reconstruction_n[i,j] - test_n[i,j]| / \sum_{i,j} test_n[i,j])$$

Equation 5: Reconstruction Accuracy

The histogram in figure 7 shows the distribution of accuracies for the images in the training dataset. The results show that the models are reconstructing some of the tree structure for most images, but there is significant error in all images and in some there is very little accurate structure captured in the model. Figure 8 shows selections from the true masks (from the dataset) and the reconstructions.

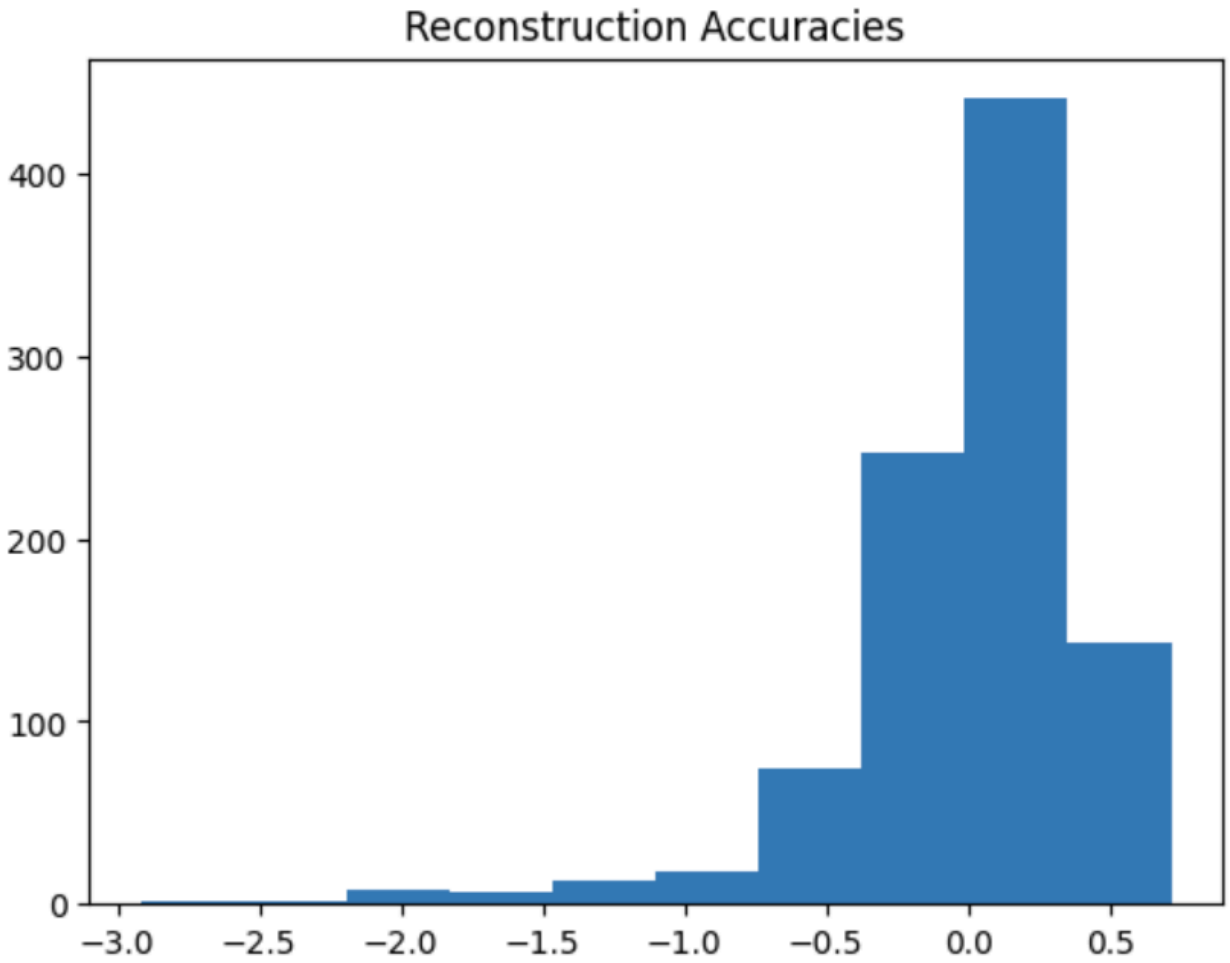


Figure 8: Distribution of accuracy scores

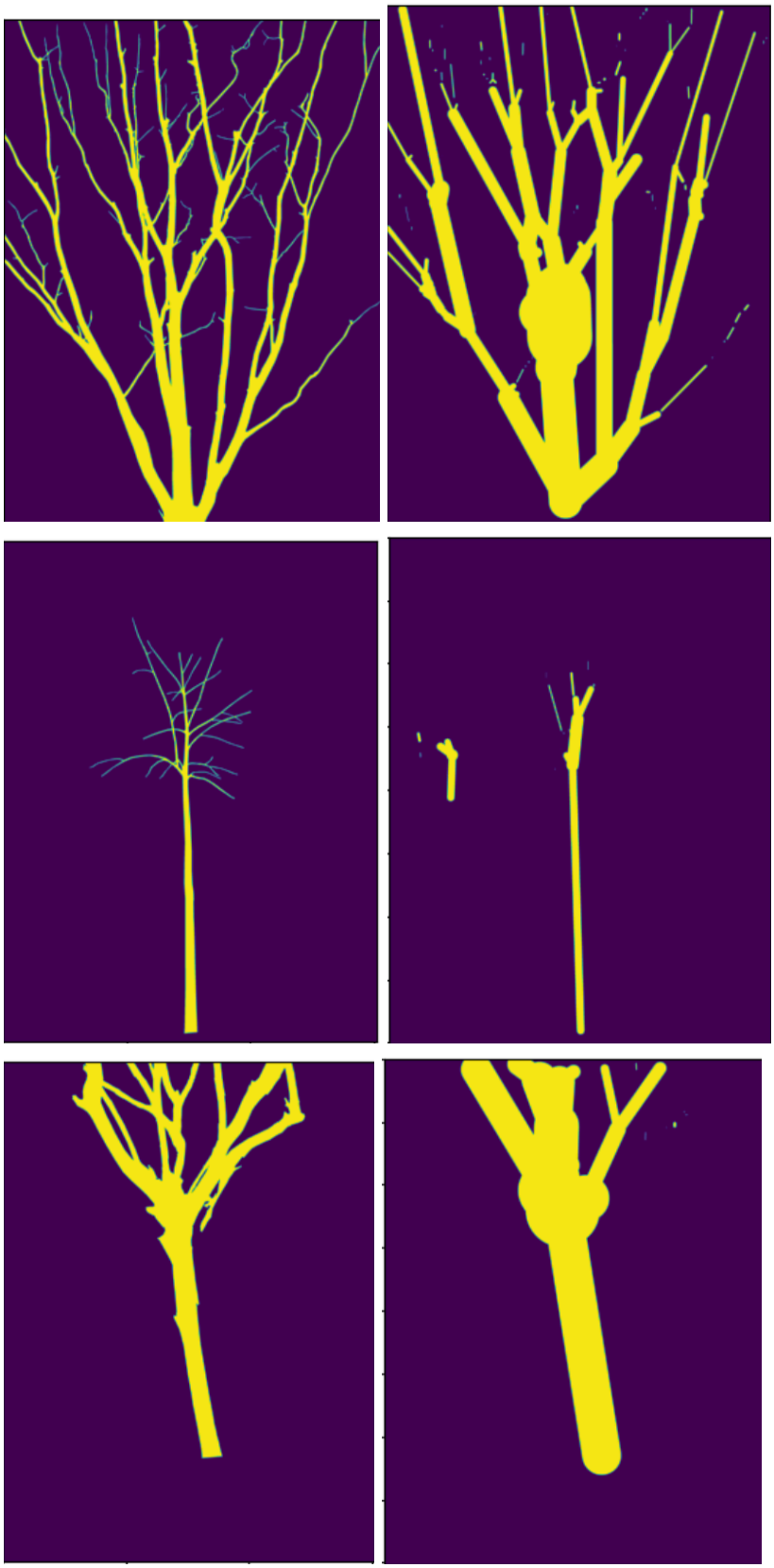


Figure 9: 3 pairs of truth value mask (left) compared to the reconstructed model (right)

The images in figure 8 show that the model struggles in a few key areas:

- Estimation of node width
- Segmentation of small branches (CNN splits them into short free-floating lines)
- Removing disconnected nodes introduced by error in the CNN output

Improving the segmentation of small branch segments could be achieved by more training, and possible by augmenting the training data (random cropping,m adding noise). The plot in figure 2 suggests that the model has not yet converged to its optimum solution. Tuning hyper parameters such as number of layers, number of channels in each layer, or learning rates could also help. However tuning hyper parameters is a time and computation intensive process that is outside the scope of this project.

A better method for finding the width of each node would also improve the results. The method used here is simple, but it drastically overestimates the width if a tree's branch is aligned with the horizontal or vertical axis. This is responsible for the large blobs in the center of many of the reconstructed masks.

Confidence Metric

To assess the value of the models self-scoring metric, the correlation between the reconstruction accuracy ($A = \{A_1, A_2, .. A_N\}$) and the confidence score ($C = \{C_1, C_2, ... C_N\}$) is calculated with a Pearson correlation (equation 6).

$Confidence\ Significance = cov(A, C) / \sigma_A * \sigma_C$ Equation 6: Confidence Score Correlation

The correlation coefficient for the models created from the training dataset was less than 0.01, which indicates that there is absolutely no correlation between the confidence score and the reconstruction accuracy. The quantiles plot in figure 9 shows this in more detail. The ideal scenario would be to see the points clustering around the line $y = x$, as that indicates a direct correlation. However that is not the case, the points are clustered pretty much everywhere else.

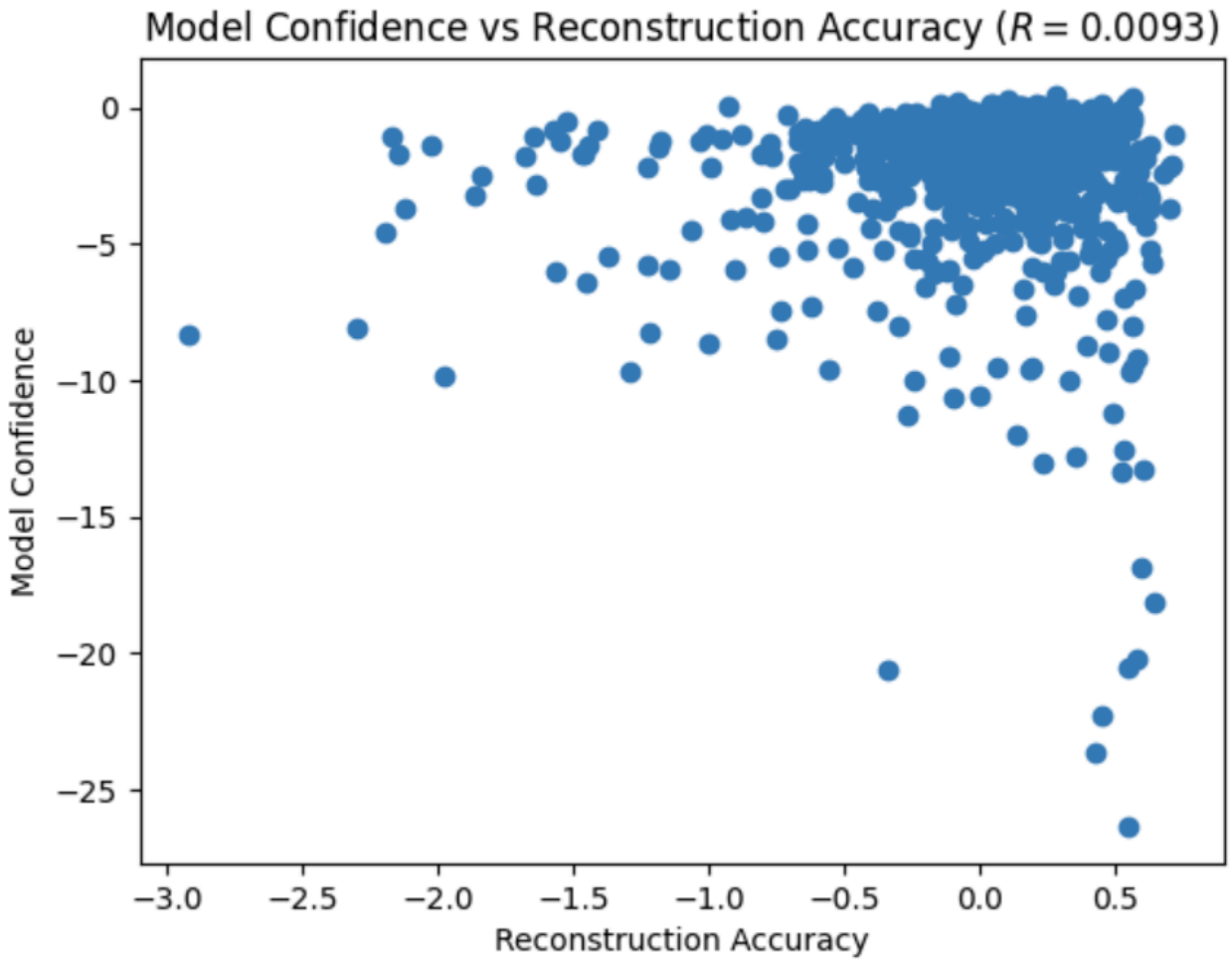


Figure 10: Model confidence vs true reconstruction accuracy

It is difficult to say what might improve the confidence score. It may be that improving the baseline reconstruction accuracy first is necessary, so the confidence score has a stable comparison.

Improving the structural information in the model could help improve the confidence score. Currently each model is a set of nodes, with no knowledge of which is the root, which are the tips. Adding that information could help to check the smoothness of the width gradient from root node to tips. It would also help “prune” floating nodes that are produced by errors in CNN output. Attempts made to add that functionality to this project resulted in recursion errors due to poor handling of loops in the skeleton image, so more development time would be needed to handle that in a more effective way.

Appendix: Relevant CNN Definitions

Channel

A channel is one dimension of information in an image. A simple example is the 3 channels of color data in an RGB image, or the 4 channels in an RGBD stereo image with color and distance. Within the neural network, channels are the activations for each of the learned filters, representing more abstract dimensions of data like “line-ness” or “face-ness”.

Minibatch

A minibatch is a set of data that is fed through the model simultaneously (this may or may not be executed truly in parallel). During training, average loss is calculated for the minibatch once all data in the minibatch has been processed and the model weights are updated based on this average loss.

Convolution

This layer type performs a convolution of a learned filter across the entire input image, essentially giving each pixel a score indicating how much it matches that filter. Convolutional layers early in the network learn simple features such as lines and corners. Deeper layers learn more abstract features, for example eyes and mouths if a network is trained to recognize human faces.

Batch Norm

This is a processing step performed to improve the training performance of the network. Each channel is transformed to a standard normal distribution, based on the statistics of the minibatch [5].

Pooling

Pooling adjusts the dimensions of an image. A pooling operation takes a neighborhood of pixels and outputs one value based on some operation upon the neighborhood, such as max, min, or mean. Max pooling is the most commonly used pooling type.

Unpooling increases an image size, producing a neighborhood of pixels from one value. It also takes indices produced from a previous pooling operation to know where to place data in the output. Figure A1 illustrates max pooling and unpooling.

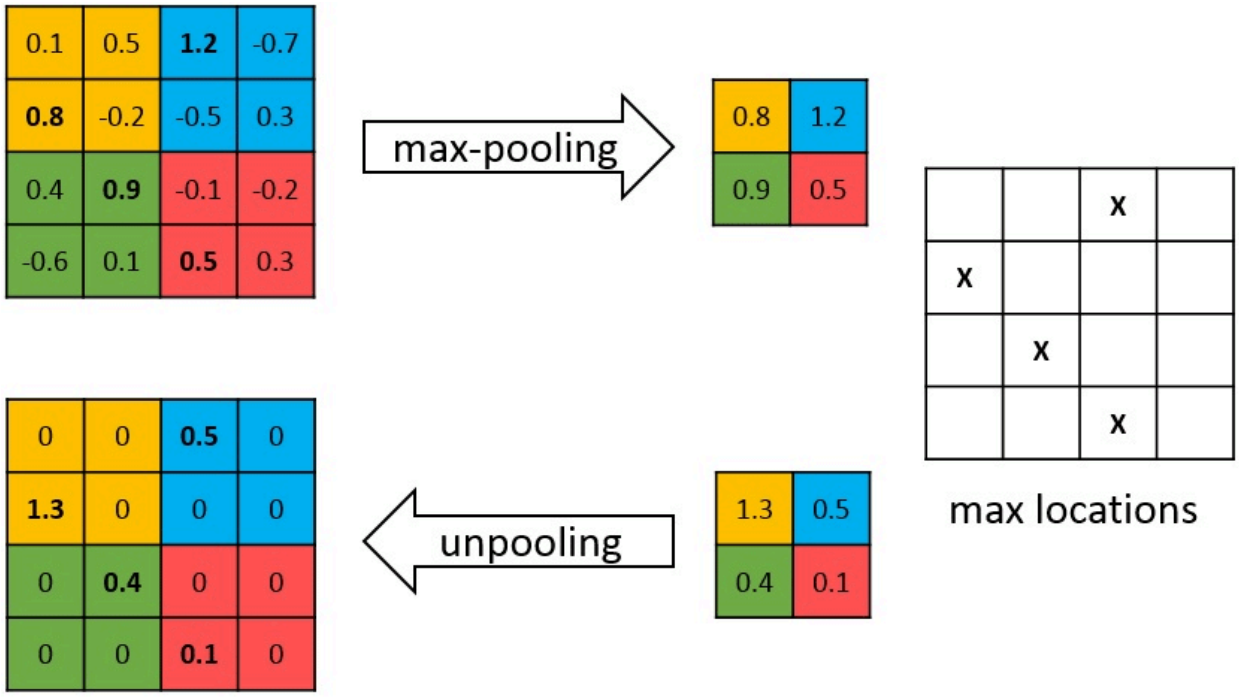


Figure A1: Max pooling operations
Source: [9]

Activation Function

Activation functions are nonlinear functions applied by a layer to the product of that layer's weights and input values. The nonlinearity of these functions is what allows neural networks to learn complex “reasoning” and conditional logic. Two common functions, and the two used in this model, are the Rectified Linear Unit (ReLU, equation A1) and the Sigmoid function (equation A2).

$f(x) = max(0, x)$ Equation A1: ReLU activation function

$f(x) = \frac{1}{1 + e^{-x}}$ Equation A2: Sigmoid activation function

References

[1] R. Verbiest, K. Ruysen, T. Vanwalleghem, E. Demeester, and K. Kellens, “Automation and robotics in the cultivation of pome fruit: Where do we stand today?,” *J. Field Robot.*, vol. 38, no. 4, pp. 513–531, Jun. 2021, doi: 10.1002/rob.22000.

[2] A. You *et al.*, “An autonomous robot for pruning modern, planar fruit trees.” arXiv, Jun. 14, 2022. Accessed: Oct. 15, 2023. [Online]. Available: http://arxiv.org/abs/2206.07201

[3] L. C. Siebert, L. F. R. B. Toledo, P. A. B. Block, D. B. Bahlke, R. A. Roncolatto, and D. P. Cerqueira, “A survey of applied robotics for tree pruning near overhead power lines,” in *Proceedings of the 2014 3rd International Conference on Applied Robotics for the Power Industry*, Oct. 2014, pp. 1–5. doi: 10.1109/CARPI.2014.7030070.

[4] A. Paszke *et al.*, “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” *Advances in Neural Information Processing Systems 32*. Curran Associates, Inc., pp. 8024–8035, 2019. Accessed: Oct. 15, 2023. [Online]. Available: http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf

[5] D. Matani, “Efficient Image Segmentation Using PyTorch: Part 2,” Medium. Accessed: Oct. 22, 2023. [Online]. Available: https://towardsdatascience.com/efficient-image-segmentation-using-pytorch-part-2-bed68cadd7c7

[6] V. Badrinarayanan, A. Kendall, and R. Cipolla, “SegNet: A Deep Convolutional Encoder-Decoder Architecture for Image Segmentation.” arXiv, Oct. 10, 2016. Accessed: Dec. 10, 2023. [Online]. Available: http://arxiv.org/abs/1511.00561

[7] T. Yang, S. Zhou, Z. Huang, A. Xu, J. Ye, and J. Yin, “Urban street tree dataset for image classification and instance segmentation,” *Comput. Electron. Agric.*, vol. 209, p. 107852, Jun. 2023, doi: 10.1016/j.compag.2023.107852.

[8] “scikit-image: image processing in Python [PeerJ].” Accessed: Dec. 11, 2023. [Online]. Available: https://peerj.com/articles/453/

[9] “(PDF) DeepPainter: Painter Classification Using Deep Convolutional Autoencoders.” Accessed: Dec. 10, 2023. [Online]. Available: https://www.researchgate.net/publication/306081538_DeepPainter_Painter_Classification_Using_Deep_Convolutional_Autoencoders